

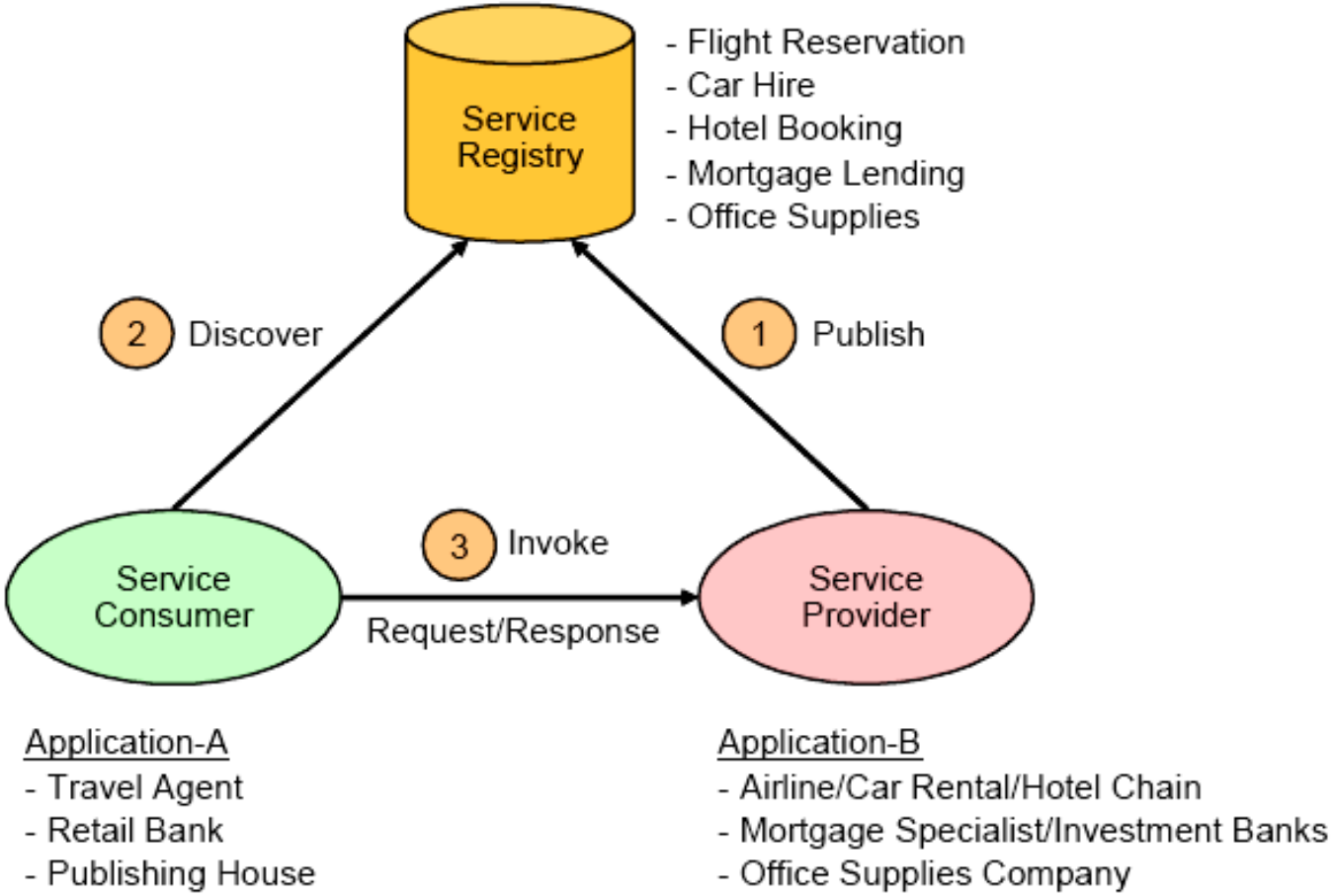
Microservice Architecture and Docker

Service-Oriented Architecture (SOA):

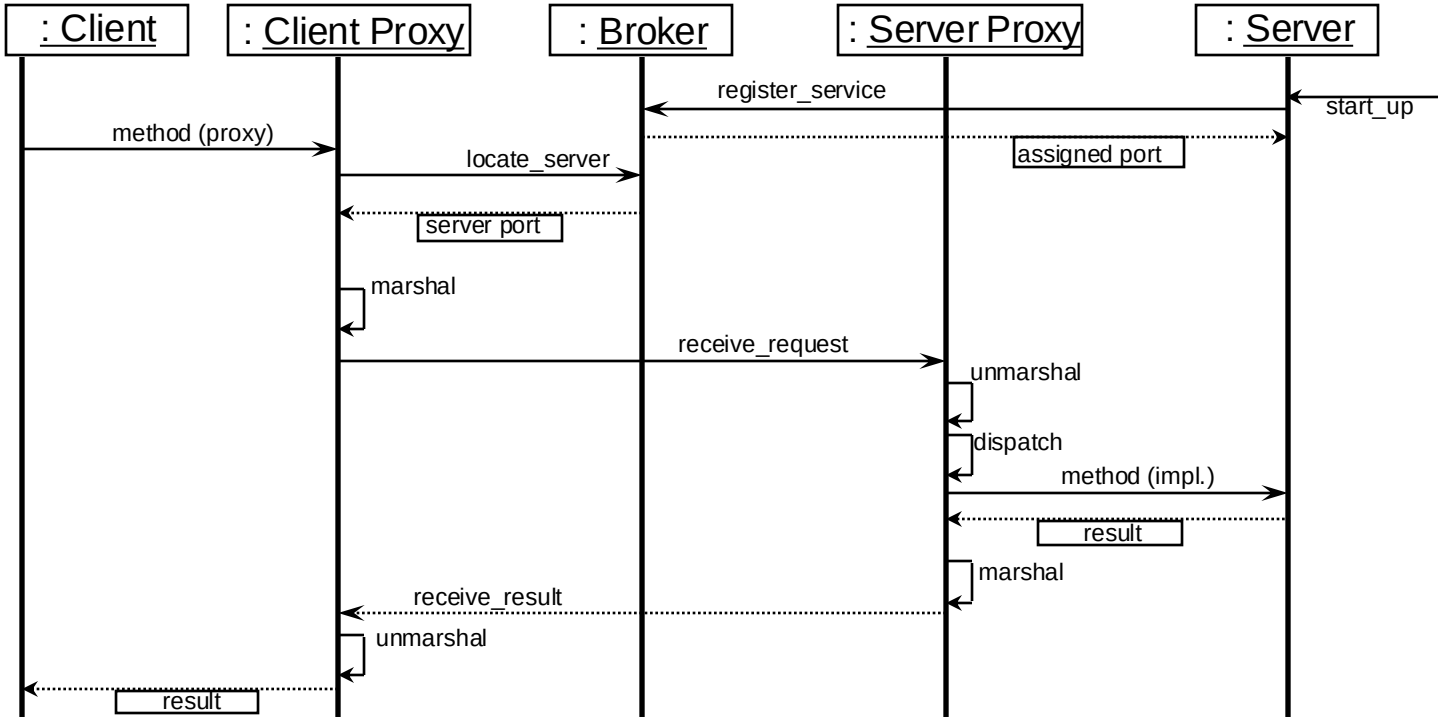
- *The policies, practices, frameworks that enable application functionality to be provided and consumed as sets of services published at a granularity relevant to the service consumer.*
- *Services can be*
  - *invoked,*
  - *published and*
  - *discovered,*

and are abstracted away from the implementation using a single, standards-based form of interface.

SOA Components and Operations



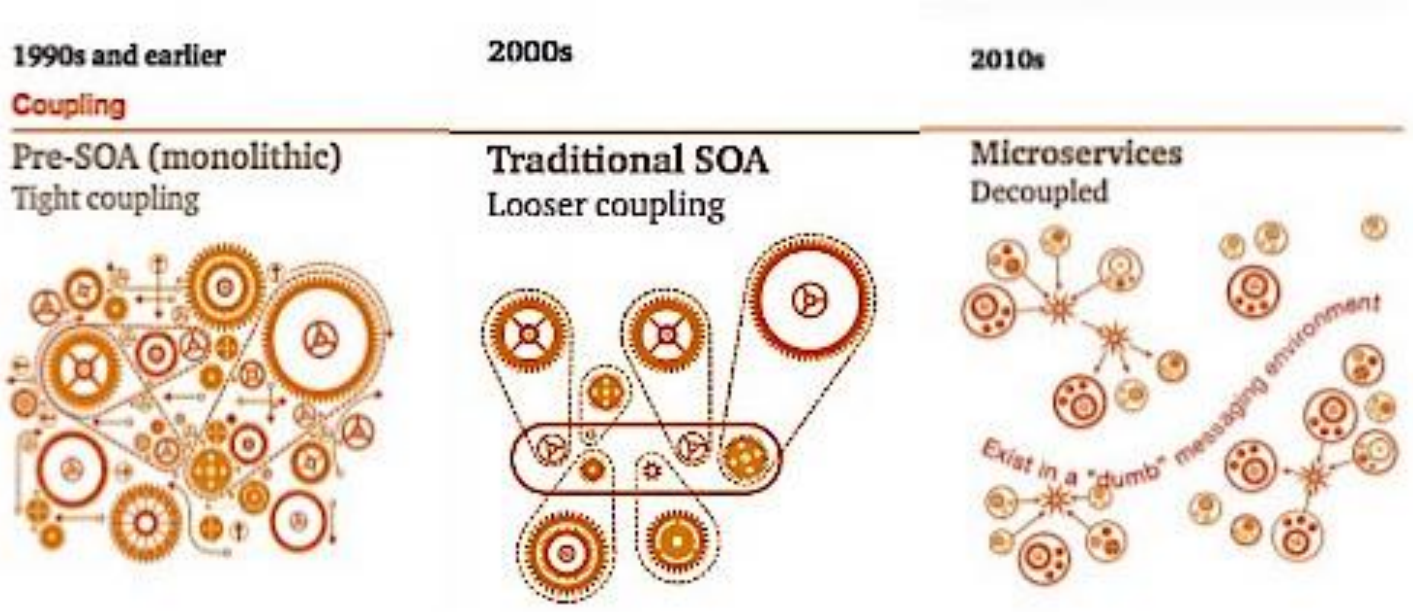
Broker Design Pattern Dynamics



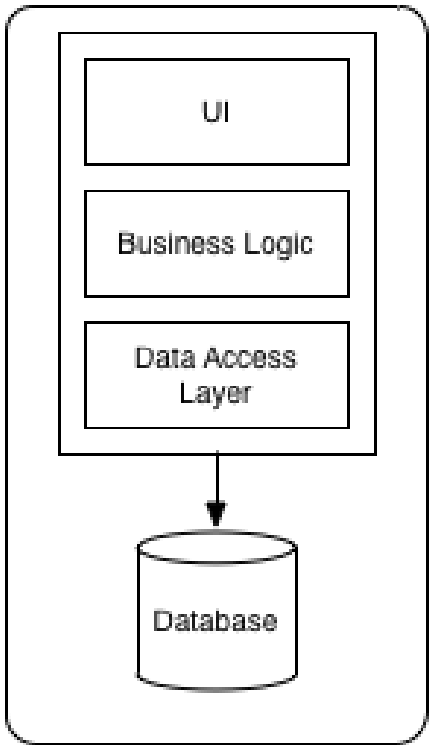
Broker tools provide the generation of necessary client & server proxies from higher level interface definitions



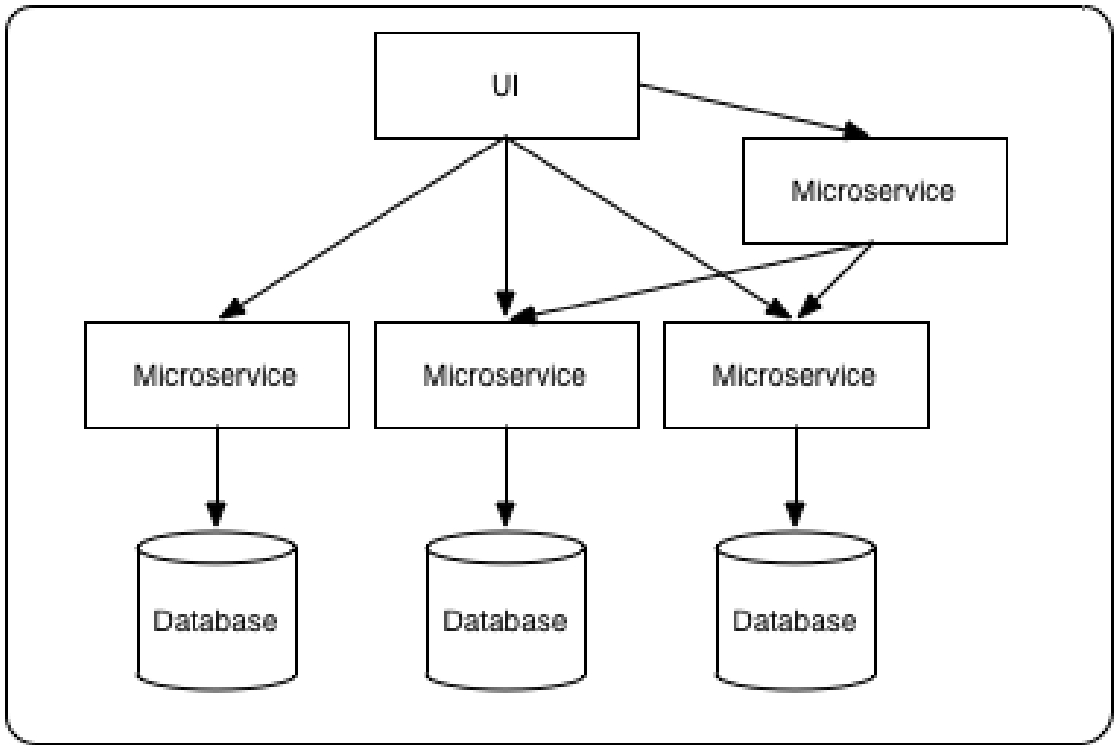
Microservices



Microservices

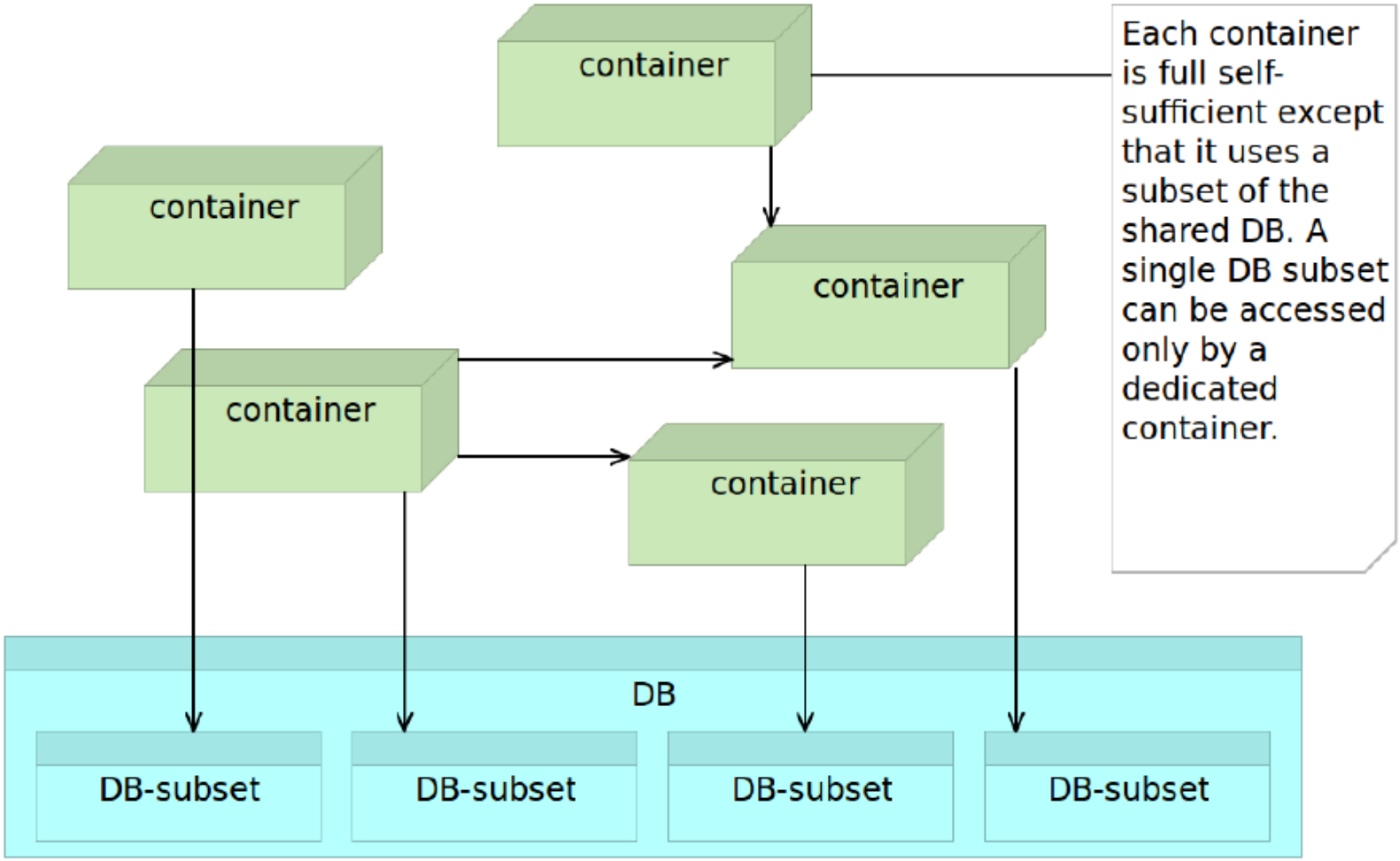


Monolithic Architecture



Microservices Architecture

Microservices Accessing the Shared Database



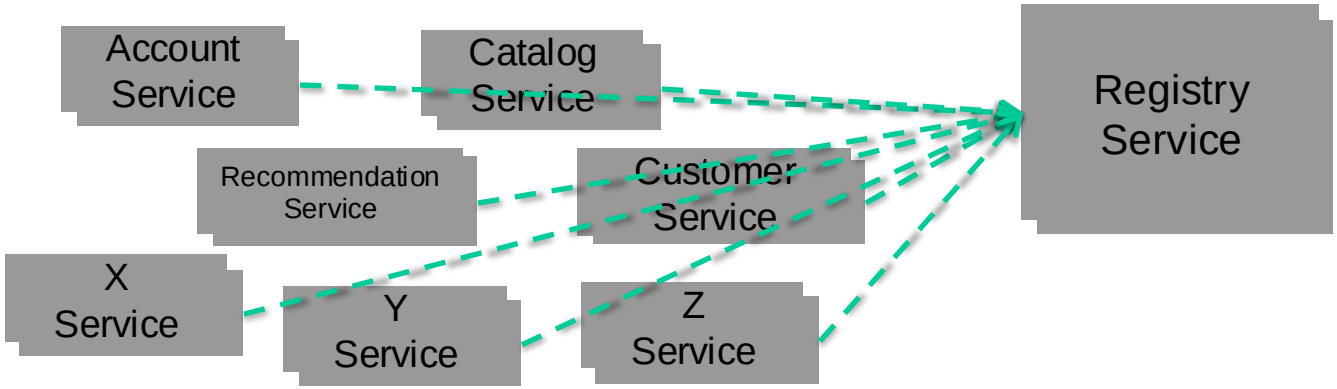
Microservices Characteristics

- Many smaller (fine grained), clearly scoped services
  - Single Responsibility Principle
  - Independently Managed
- Clear ownership for each service
  - Typically need/adopt the “DevOps” model



Service Discovery

- 100s of MicroServices
  - Need a Service Metadata Registry (Discovery Service)

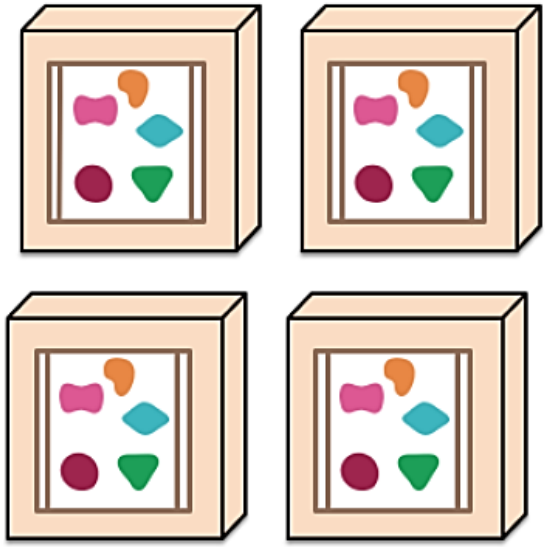


Microservices. Scalability

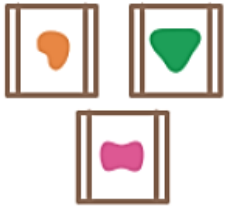
A monolithic application puts all its functionality into a single process...



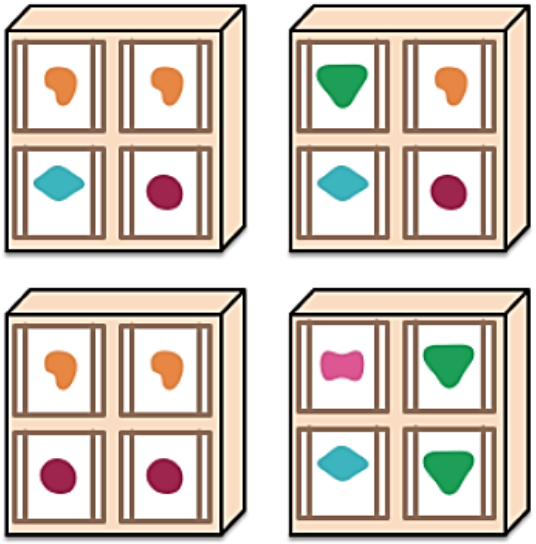
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



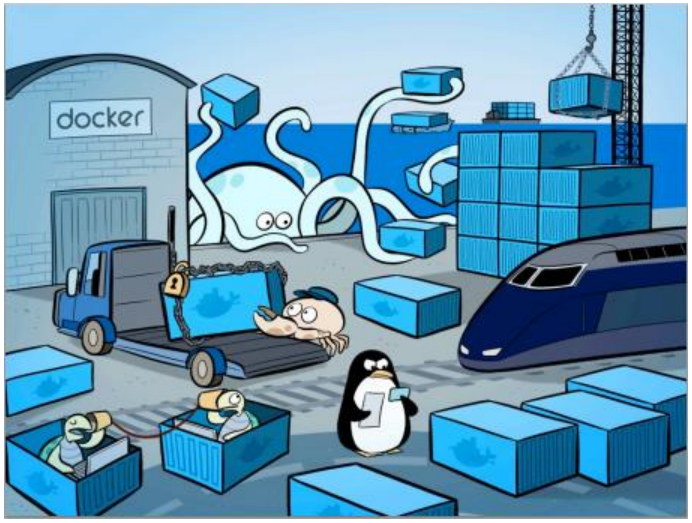
... and scales by distributing these services across servers, replicating as needed.



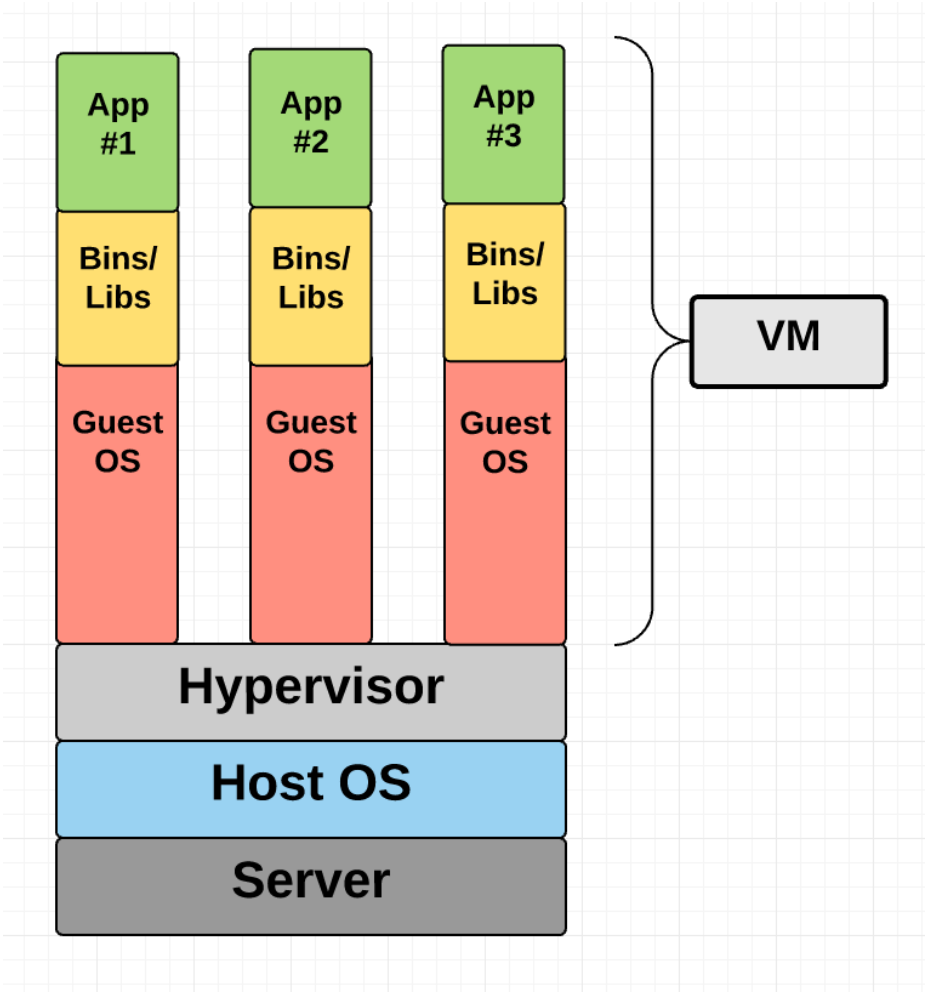
Docker: Containerization for Software



Docker

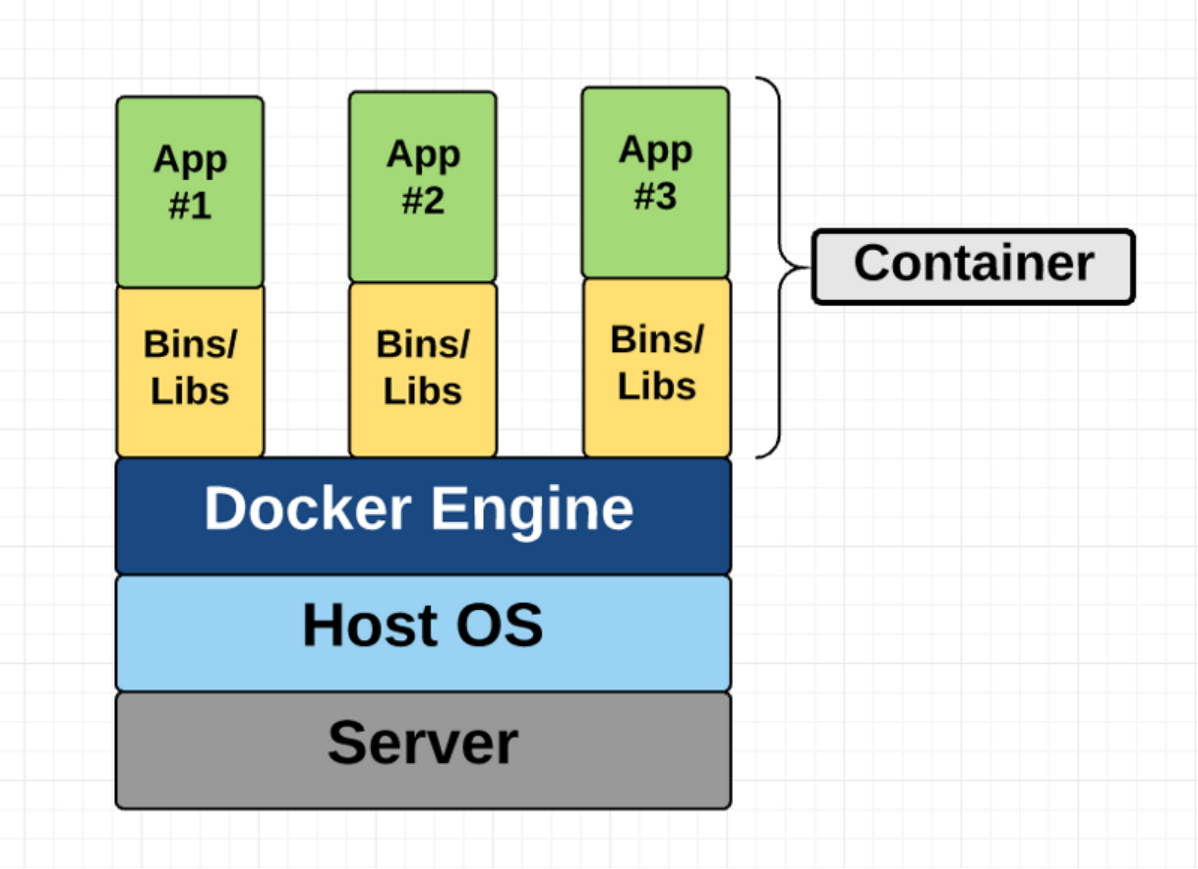


“Docker is an open platform for developers and sysadmins to build, ship, and run distributed applications”



Virtual Machine

Container


App #1
App #2
App #3
Bins/ Libs
Bins/ Libs
Bins/ Libs
Container
Docker Engine
Host OS
Server



So why Docker?

- Containers are far from new;
  - Google has been using their own container technology for years.
  - Others Linux container technologies include
    - Solaris Zones,
    - BSD jails, and
    - LXC, which have been around for many years.
- Docker is an open-source project based on Linux containers. It uses Linux Kernel features.



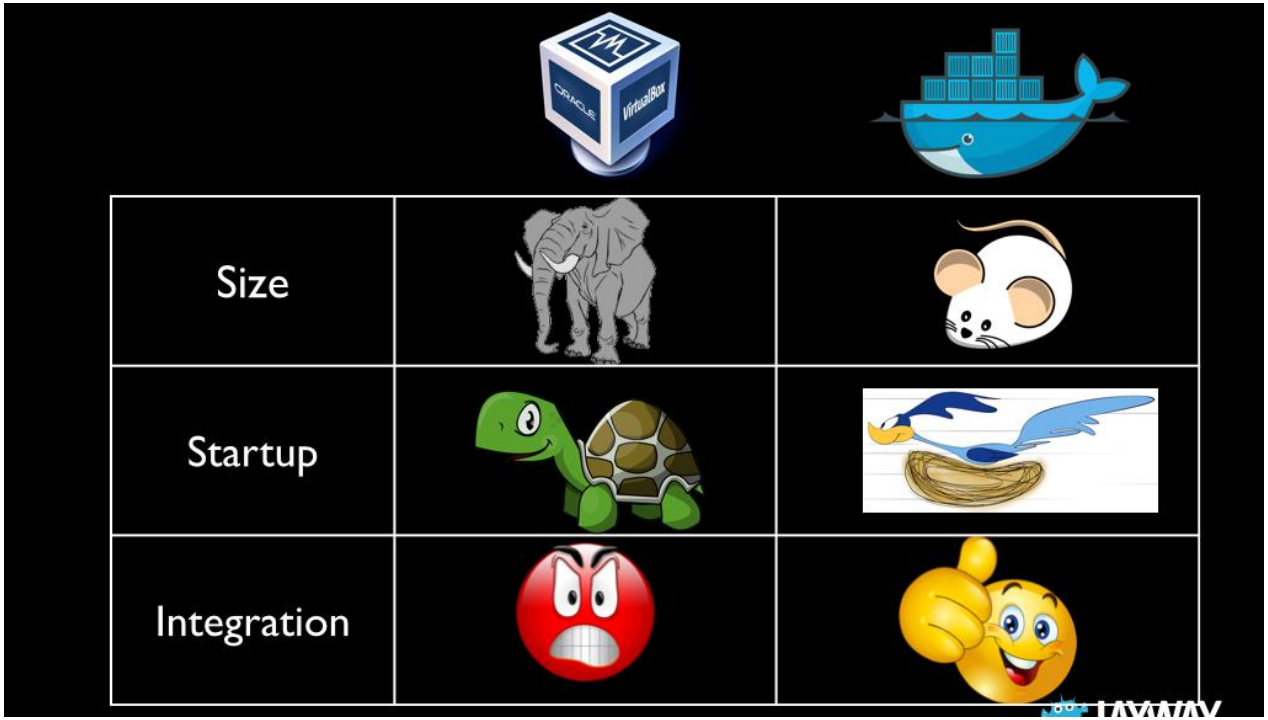
Docker Benefits

1. Local development environments can be set up that are exact replicas of a live environment/server.
2. It simplifies collaboration by allowing anyone to work on the same project with the same settings, irrespective of the local host environment.
3. Multiple development environments can be run from the same host each one having different configurations, operating systems, and software.
4. Projects can be tested on different servers.
5. It gives you instant application portability. Build, ship, and run any application as a portable container that can run almost anywhere.

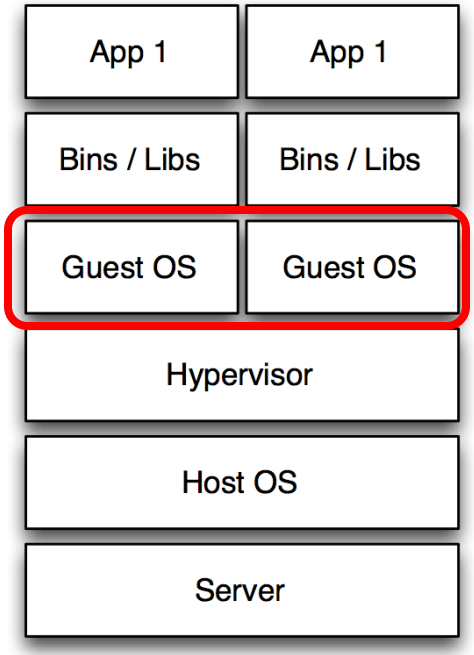
Why Docker?

- **Ease of use.** It allows anyone to package an application on their laptop, which in turn can run unmodified anywhere
  - The mantra is: “build once, run anywhere.”
- **Speed.** Docker containers are very lightweight and fast. Since containers are just sandboxed environments running on the kernel, they take up fewer resources. You can create and run a Docker container in seconds, compared to VMs which might take longer because they have to boot up a full virtual operating system every time.
- **Docker Hub.** Docker users also benefit from the increasingly rich ecosystem of Docker Hub, which you can think of as an “app store for Docker images.” Docker Hub has tens of thousands of public images created by the community that are readily available for use.
- **Modularity and Scalability.** Docker makes it easy to break out your application’s functionality into individual containers. With Docker, it’s become easier to link containers together to create your application, making it easy to scale or update components independently in the future.

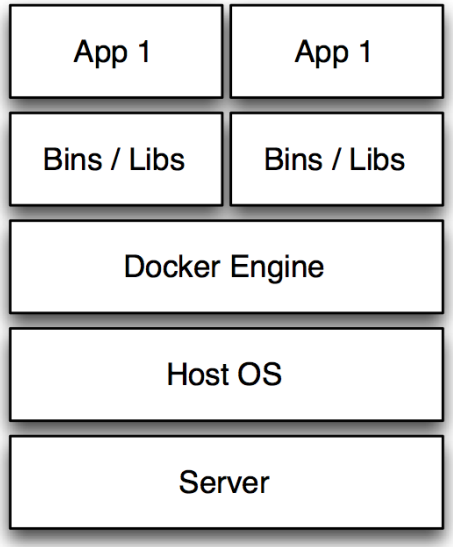
VM vs. Docker



VM vs. Docker (Containers)

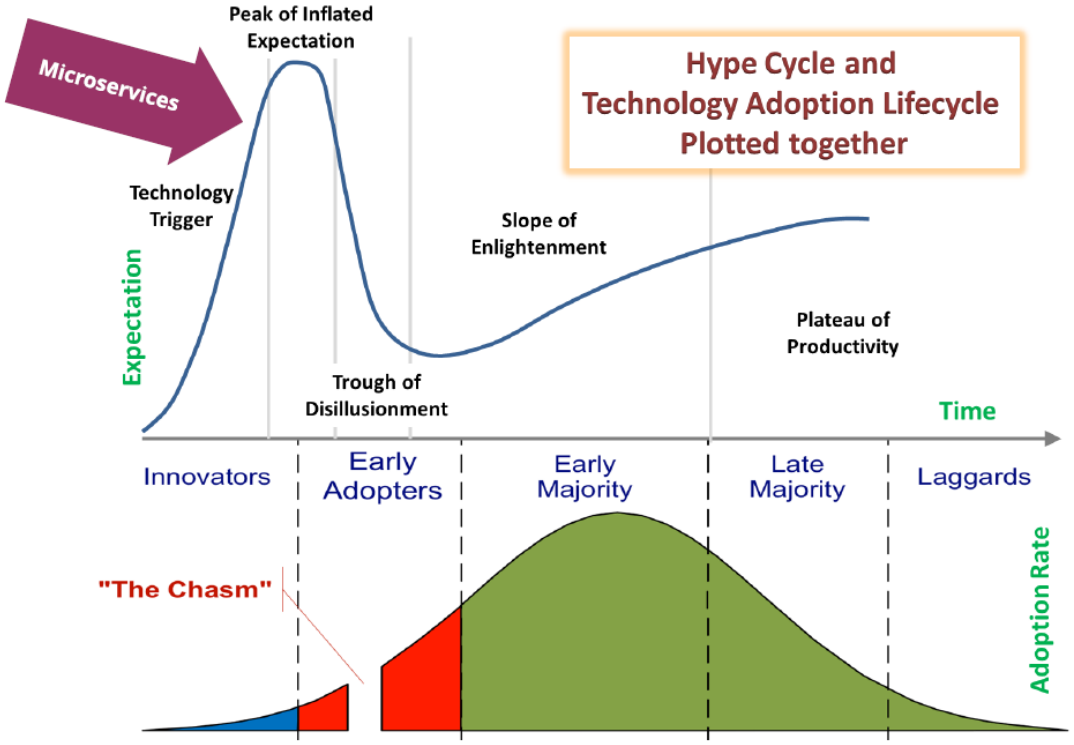


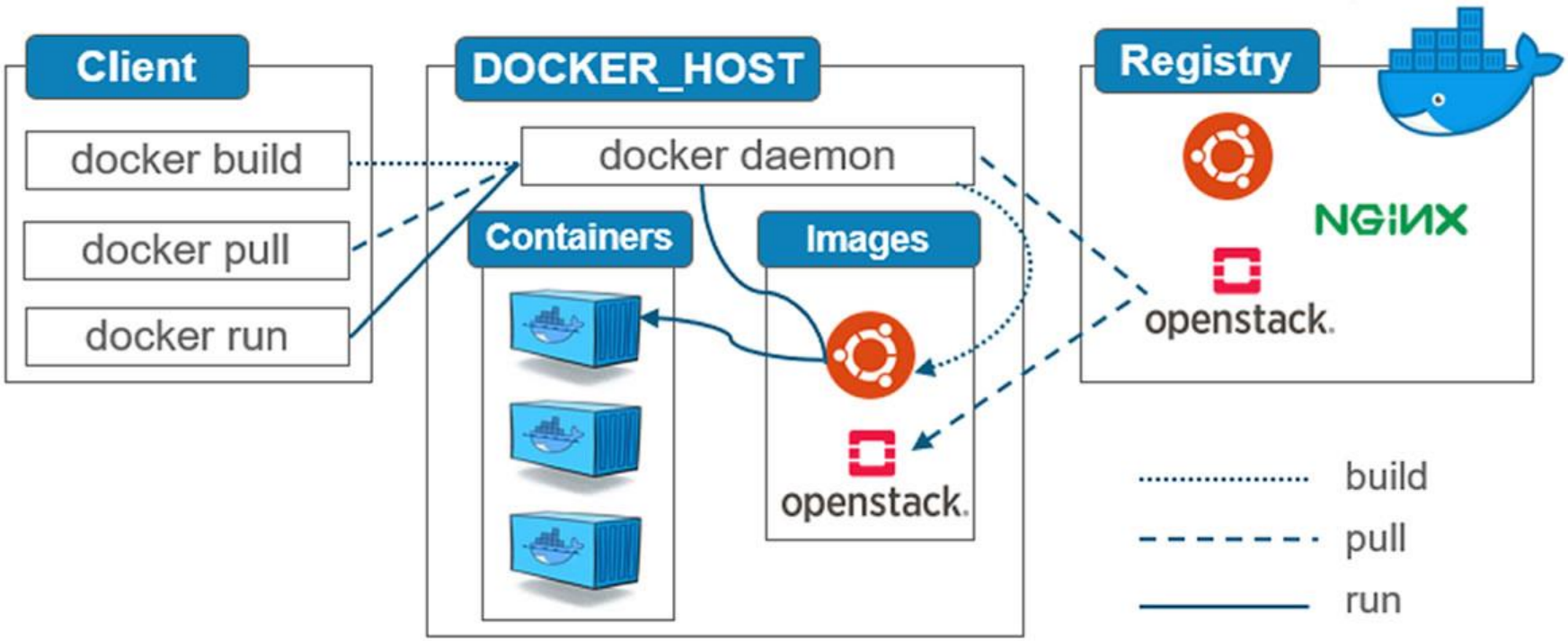
Virtual Machines

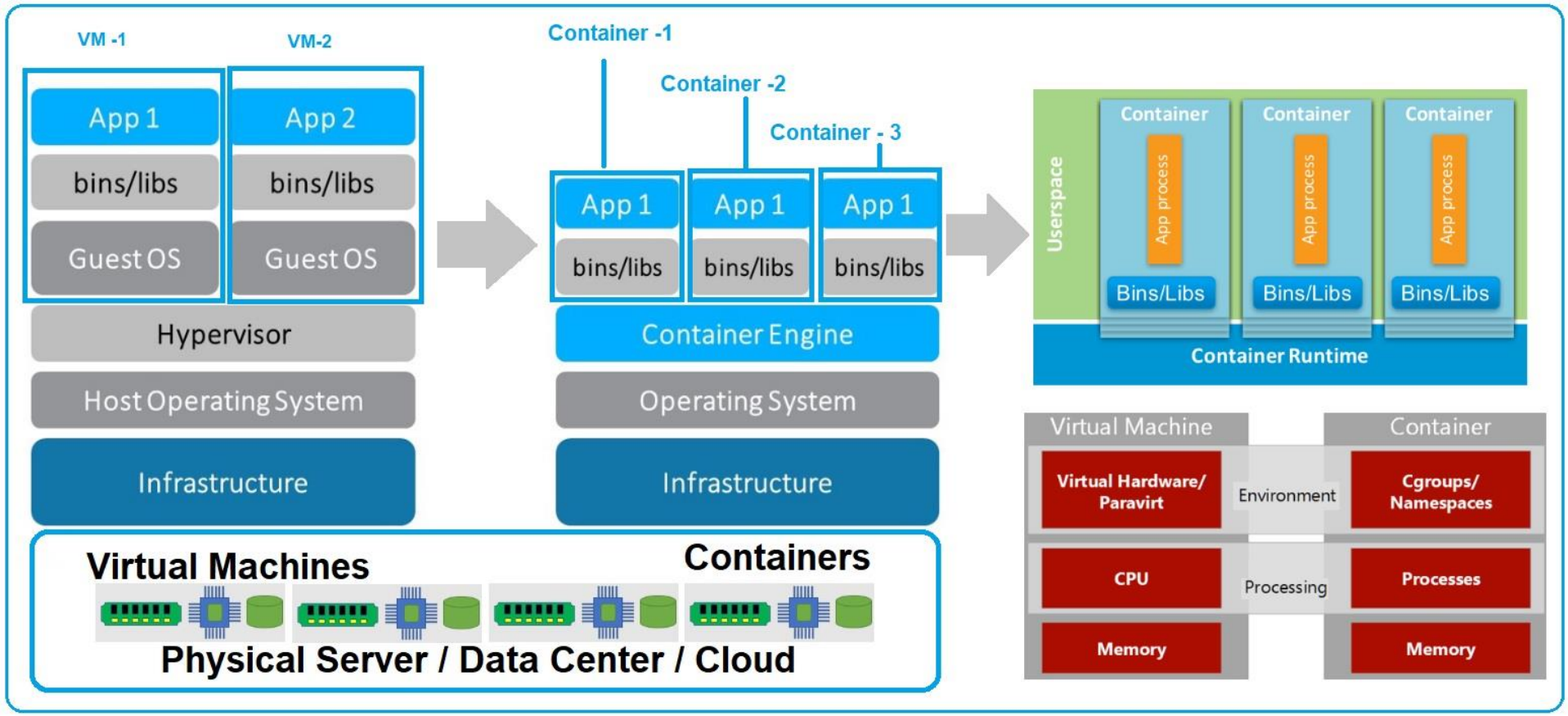


Docker

Gartner Hype Cycle




Client
docker build
docker pull
docker run
DOCKER_HOST
docker daemon
Containers
Images
openstack.
Registry
NGINX
openstack.
build
pull
run


VM -1
VM-2
App 1
App 2
bins/libs
bins/libs
Guest OS
Guest OS
Hypervisor
Host Operating System
Infrastructure
Container -1
Container -2
Container - 3
App 1
App 1
App 1
bins/libs
bins/libs
bins/libs
Container Engine
Operating System
Infrastructure
Virtual Machines
Containers
Physical Server / Data Center / Cloud
Userspace
Container
Container
Container
App process
App process
App process
Bins/Libs
Bins/Libs
Bins/Libs
Container Runtime
Virtual Machine
Container
Virtual Hardware/ Paravirt
Environment
Cgroups/ Namespaces
CPU
Processing
Processes
Memory
Memory